

DOKUMEN ARSITEKTUR JARINGAN DAN PROTOKOL MANAJEMEN KUNCI
IPB SPACE: CAMPUS FACILITY & QUEUE SYSTEM
SPESIFIKASI TOPOLOGI INFRASTRUKTUR DAN INTEGRASI ALUR KERJA
PROTOKOL AAA



Disusun Oleh:

KELOMPOK 4

1. Muhammad Farhadh (G6401231080)
2. Naufal Ghifari Afdhala (G6401231029)
3. Samuel Christian (G6401231037)

DEPARTEMEN ILMU KOMPUTER
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
INSTITUT PERTANIAN BOGOR
BOGOR
2026

INFORMASI DOKUMEN DAN METADATA KEAMANAN

Parameter Arsitektur	Spesifikasi Protokol Komunikasi dan Manajemen Kunci
Mata Kuliah	KOM1315 Keamanan Informasi (PBL Terintegrasi)
Nama Dokumen Luaran	Architecture_Diagram.pdf (Protokol Manajemen Kunci & Komunikasi)
Nama Sistem Teruji	IPB Campus Facility & Queue System: IPB Space
Lokasi Folder Repositori	/02_Design_Documents/Architecture_Diagram.pdf
Format Nama Repo	KOM1315_SmtGenap26_Kelompok04_IPBSpace
Komponen Infrastruktur	React.js (Vercel Client), FastAPI (Railway Server), PostgreSQL (Railway Database), Appwrite Storage (Cloud Bucket), dan SMTP Gateway (Mailer)
Mekanisme Keamanan Jalur	Enkripsi HTTPS / TLS 1.3 (Data in Transit), Isolasi Kunci Lokal (.env Config), dan Private Database Connection Tunnel
Implementasi Protokol AAA	Bearer Token JWT Authentication, Role-Based Access Control (Authorization), dan Immutable Audit Trail PostgreSQL (Accounting)
Primitif Kriptografi Aktif	Bcrypt Password Hashing (One-Way Salted Hash) dan Column-Level Encryption (AES-256) untuk Kolom PII
Akun Akses Reviewer (Dosen)	shelvie.neyman@gmail.com & endang.p.giri@gmail.com

BAB 1

PENDAHULUAN

1.1 Latar Belakang Desain Arsitektur Keamanan

Membangun platform informasi berskala besar di lingkungan kampus seperti IPB Space memerlukan sebuah cetak biru arsitektur jaringan yang tidak hanya andal dalam memproses transaksi operasional bisnis, melainkan juga tangguh dalam mempertahankan integritas data dari interseptasi pihak luar. Ketiadaan dokumentasi arsitektur komunikasi terenkripsi dapat memicu terjadinya kesalahan konfigurasi (*security misconfiguration*), seperti terbukanya port basis data ke jaringan publik atau tidak terenskripsinya jalur komunikasi data. Oleh karena itu, diperlukan perancangan dokumen arsitektur protokol keamanan yang jelas.

1.2 Tujuan Arsitektur dan Manajemen Kunci

Dokumen ini bertujuan untuk memetakan jalur komunikasi antar-komponen platform IPB Space serta menspesifikasikan mekanisme pengelolaan parameter rahasia sistem (*cryptographic keys*) guna menjamin kerahasiaan (*confidentiality*), integritas (*integrity*), dan ketersediaan (*availability*) data sensitif universitas.

1.3 Ruang Lingkup Protokol Komunikasi

Ruang lingkup pengamanan ini mencakup seluruh interaksi data elektronik yang terjadi di dalam ekosistem IPB Space, mulai dari sisi klien (*browser* pengguna), server API *backend*, hingga kluster basis data permanen dan *cloud storage* berkas permohonan.

BAB 2

ARSITEKTUR DAN TOPOLOGI KOMUNIKASI SISTEM

2.1 Komponen Infrastruktur Aplikasi (Topologi Terdistribusi)

Arsitektur sistem informasi IPB Space dibangun dengan mengintegrasikan lima komponen utama yang terdistribusi secara modular untuk memisahkan tanggung jawab operasional dan meningkatkan ketahanan sistem:

1. **React.js Frontend Client:** Antarmuka pengguna yang berjalan pada browser civitas akademika atau administrator (di-host pada Vercel).
2. **FastAPI Backend Server:** Bertindak sebagai *API Gateway* pusat yang memproses seluruh logika bisnis secara asinkron (di-host pada Railway Cloud Platform).
3. **PostgreSQL Database Cluster:** Penyimpanan permanen data relasional untuk tabel pengguna, reservasi, antrean, dan log audit (Railway Managed Database).
4. **Appwrite Cloud Storage Bucket:** Layanan penyimpanan objek terisolasi pihak ketiga untuk mengamankan berkas digital PDF surat permohonan.
5. **SMTP Mailer Gateway:** Layanan pengiriman notifikasi surat elektronik otomatis untuk alur penawaran slot antrean.

2.2 Protokol Komunikasi dan Pengamanan Data dalam Perjalanan (*Data in Transit*)

Setiap komponen berkomunikasi menggunakan jalur enkripsi untuk memitigasi serangan penyadapan paket data rahasia:

- **Komunikasi Klien ke Server (HTTPS/TLS 1.3):** Seluruh pertukaran data dari React.js menuju REST API FastAPI diwajibkan melewati protokol **HTTPS** terenkripsi **Transport Layer Security (TLS 1.3)** untuk melindungi token akses JWT dan data privat dari risiko intersepsi *Man-in-the-Middle* (MitM).
- **Komunikasi Server ke Basis Data (Private Database Tunnel):** Akses data dari FastAPI ke PostgreSQL dilakukan menggunakan string koneksi asinkron terisolasi (postgresql+asyncpg://). Port basis data dikonfigurasi agar tertutup dari akses luar dan hanya menerima request dari alamat IP internal server *backend*.

BAB 3

PROTOKOL MANAJEMEN KUNCI DAN KRIPTOGRAFI

3.1 Isolasi Kunci Rahasia Sesi Informasi (.env & config.py)

Sesuai instruksi laboratorium Keamanan Informasi, seluruh kunci kriptografi (*hardcoded keys*) dilarang keras ditaruh di dalam repositori kode. Sistem IPB Space mengisolasi variabel rahasia ke dalam berkas konfigurasi lokal bernama `.env` yang dimuat secara otomatis oleh kelas `Settings` pada file `backend/app/core/config.py`. Berkas `.env` dimasukkan ke dalam daftar `.gitignore` agar terisolasi dari proses *push* repositori. Variabel tersebut meliputi:

- `SECRET_KEY`: String acak berkekuatan tinggi yang digunakan untuk menandatangani token sesi JWT.
- `APPWRITE_API_KEY`: Kunci otorisasi privat untuk mengamankan bucket penyimpanan dokumen.

3.2 Protokol Manajemen Kunci Enkripsi Kolom AES-256

Proses enkripsi tingkat kolom (*Column-Level Encryption*) pada PostgreSQL menggunakan kunci simetrik berbasis algoritma **AES-256**. Kunci ini dikelola langsung di sisi memori server *backend* FastAPI. Server *backend* bertindak sebagai entitas tunggal yang memegang otoritas untuk memanggil fungsi enkripsi sebelum penulisan ke disk database (*Encryption at Rest*).

3.3 Manajemen Kunci Hashing Kredensial Pengguna (*Password Hashing*)

Kata sandi pengguna wajib dilindungi menggunakan fungsi satu arah kriptografi menggunakan algoritma **Bcrypt** (dikonfigurasi melalui `pwd_context` di file `backend/app/core/security.py`). Proses penyerpihan otomatis (*Salting*) diterapkan untuk menghasilkan string hash yang unik, sehingga database kebal terhadap ancaman pembongkaran berbasis serangan *Rainbow Table* jika terjadi kebocoran basis data.

BAB 4

INTEGRASI ALUR KERJA PROTOKOL AAA (ALUR UTAMA)

4.1 Alur Aliran Autentikasi (*Authentication Workflow*)

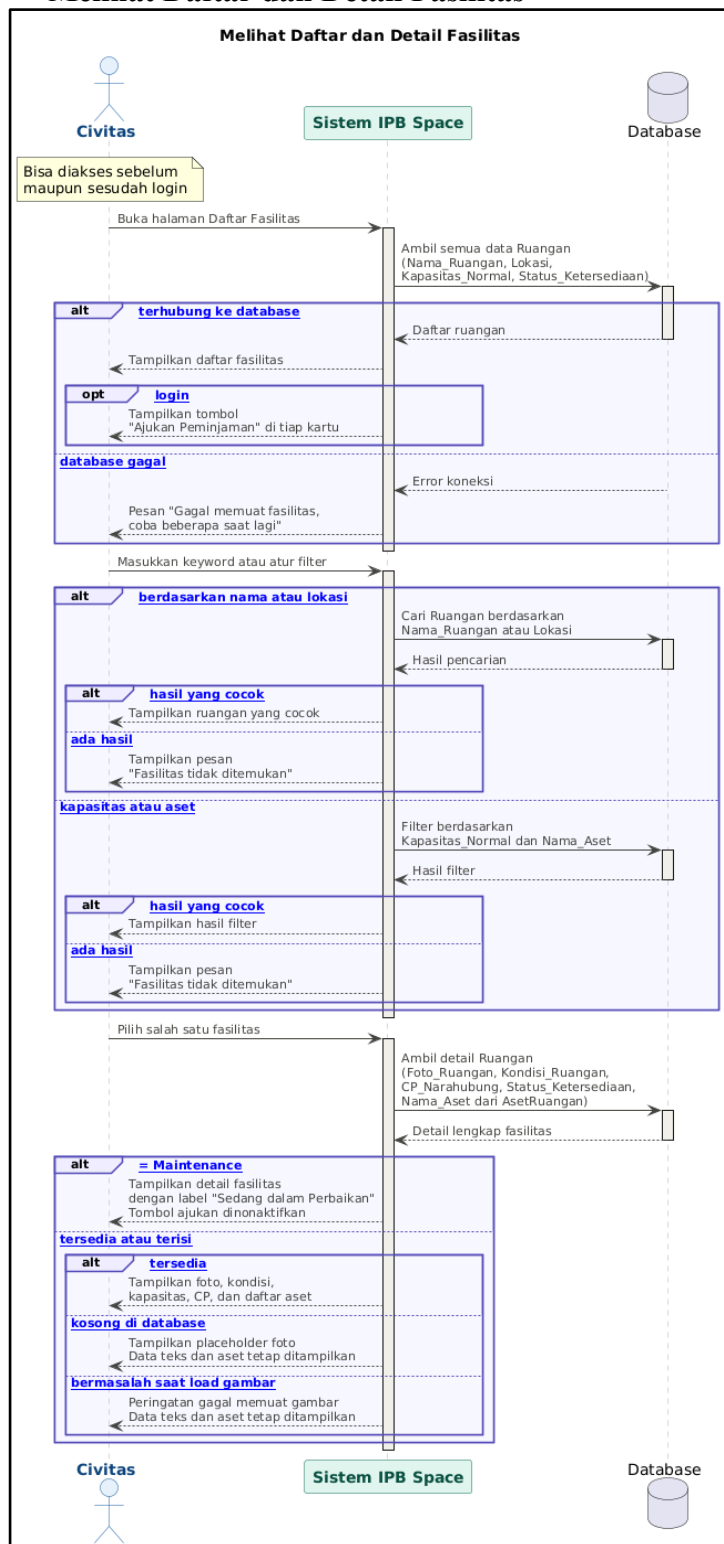
Proses verifikasi identitas civitas dijalankan melalui mekanisme *Bearer Token Validation*:

1. Pengguna menginput *email* dan kata sandi pada halaman login React.js.
2. Data dikirim via HTTPS menuju endpoint `/api/auth/login` di backend FastAPI.
3. Backend memeriksa validitas kredensial. Jika sesuai, fungsi `create_access_token` pada `app/core/security.py` akan menghasilkan string JSON Web Token (JWT) yang memuat klaim identitas (`sub`) dan waktu kedaluwarsa token.
4. Token dikembalikan ke klien untuk dilampirkan pada header `Authorization: Bearer <token>` pada setiap request transaksi fungsional berikutnya.

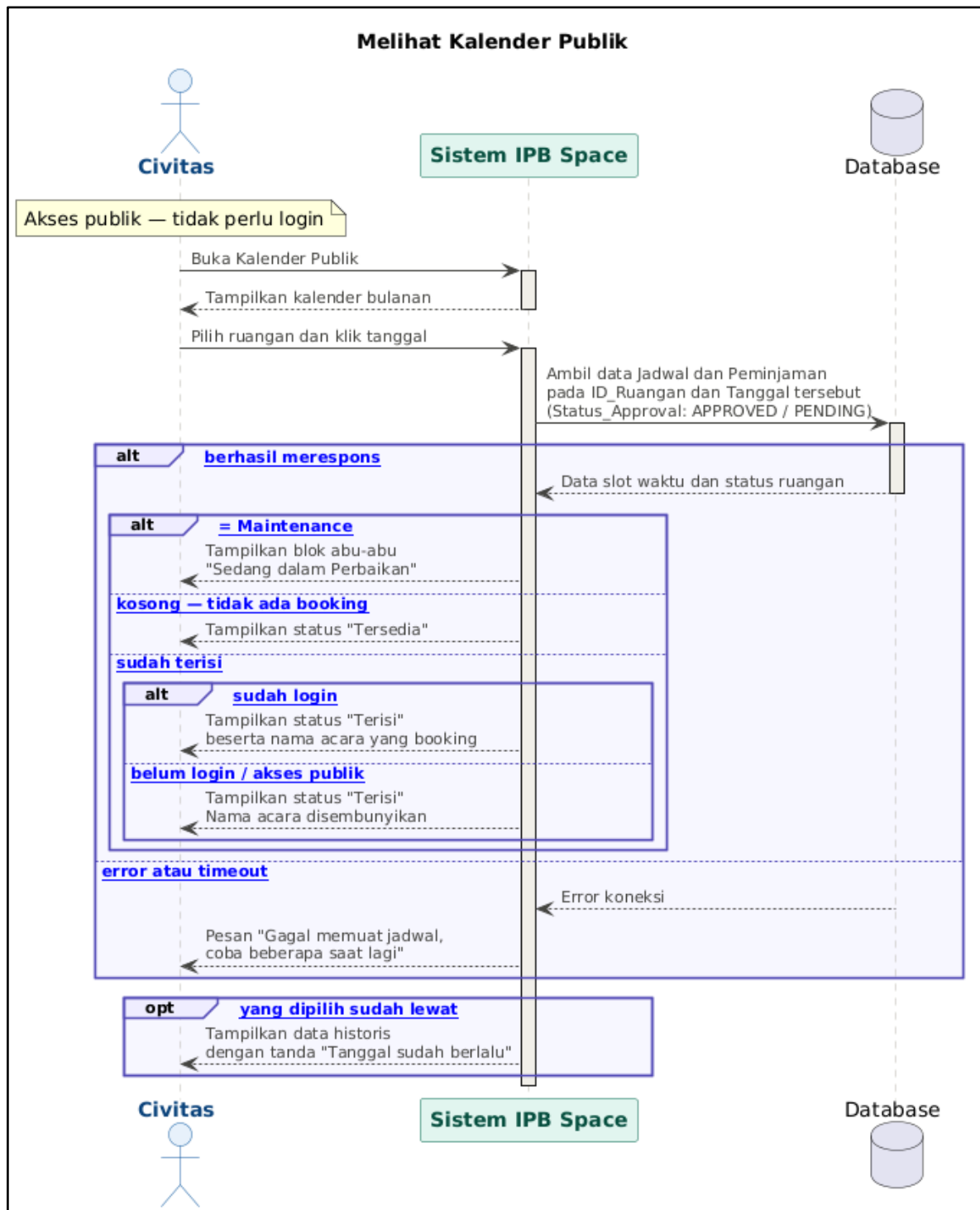
SEQUENCE DIAGRAM

1. Civitas

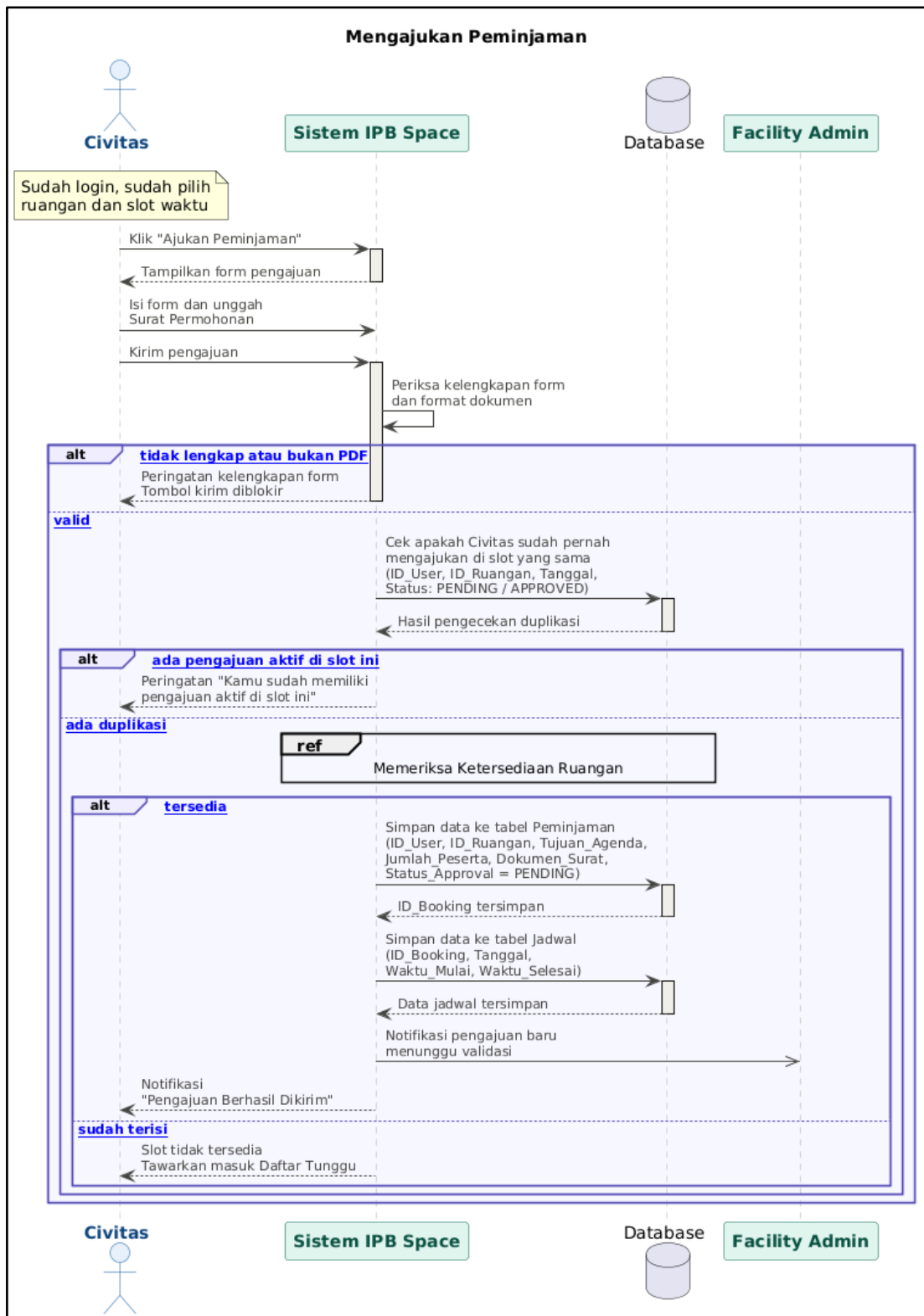
• Melihat Daftar dan Detail Fasilitas



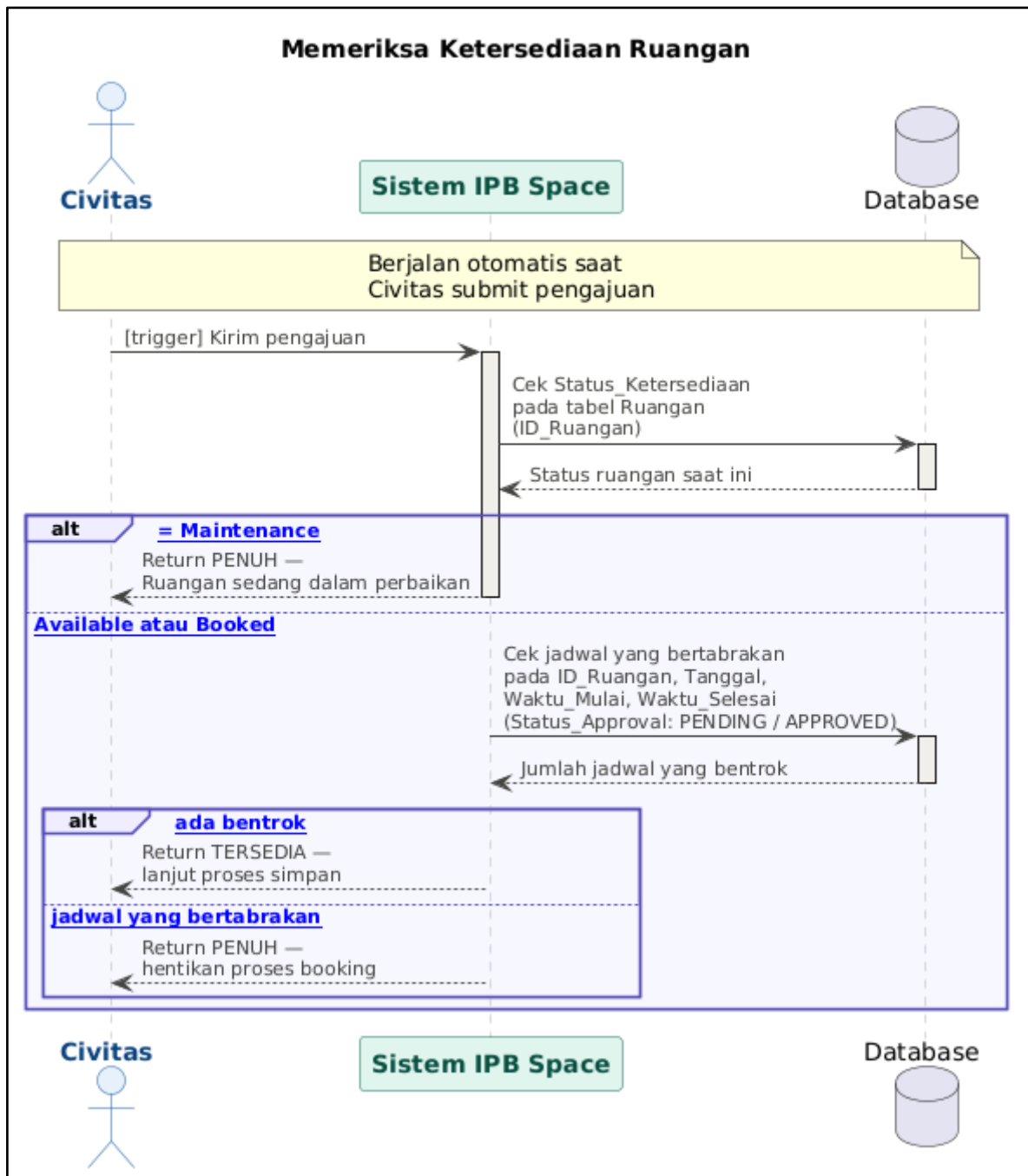
- **Melihat Kalender Publik**



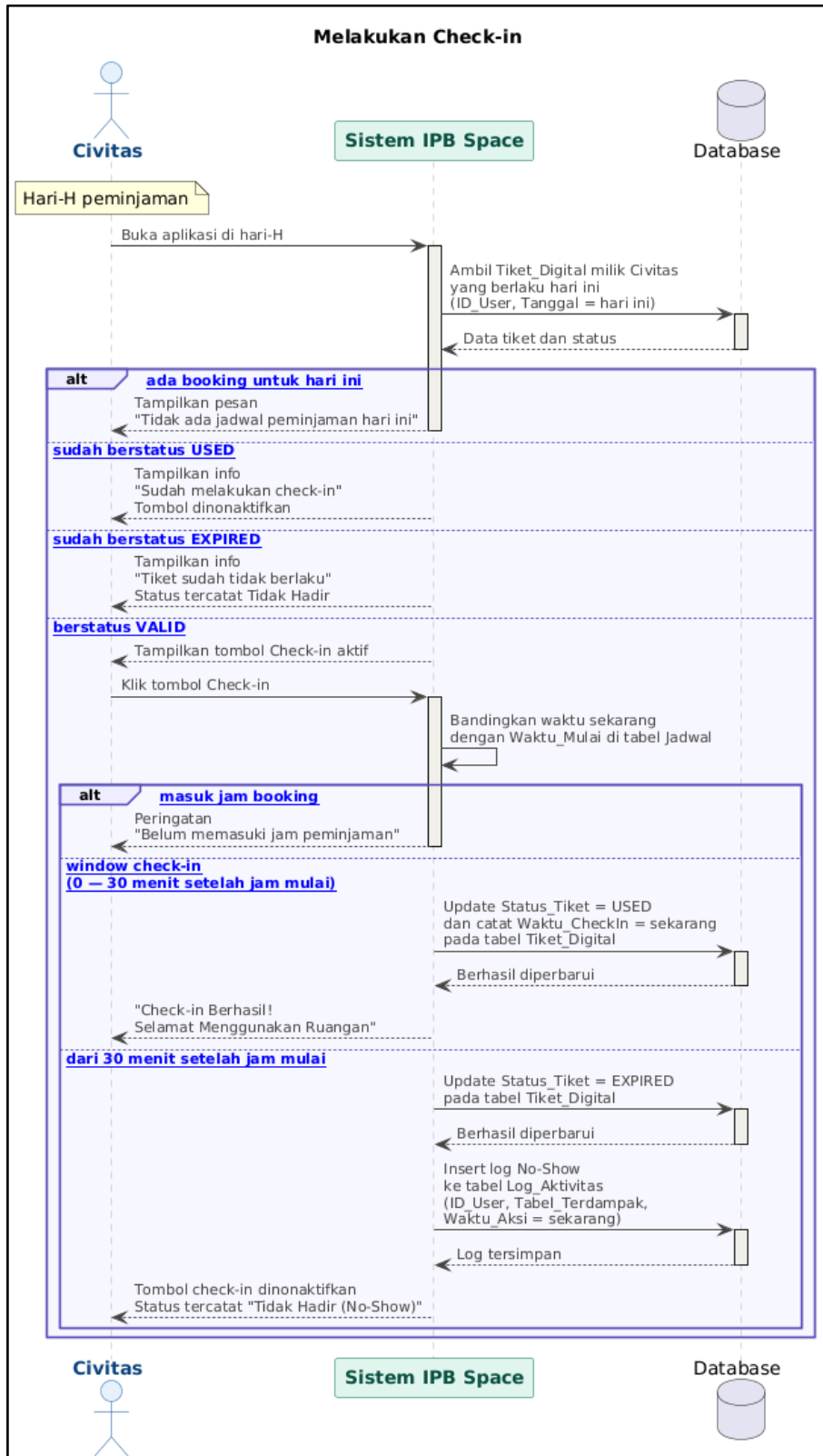
- Mengajukan Peminjaman



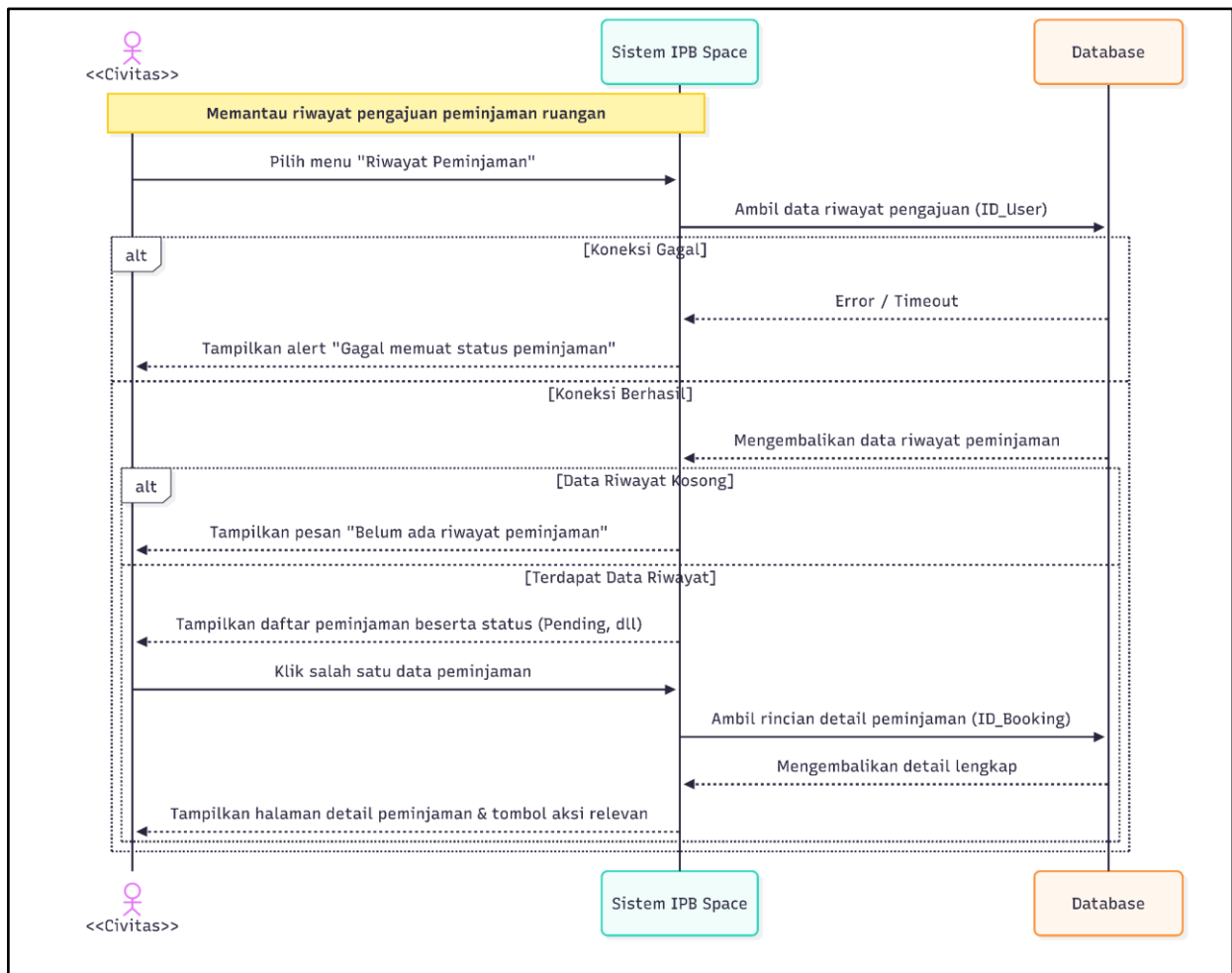
- Memeriksa Ketersediaan Ruangan



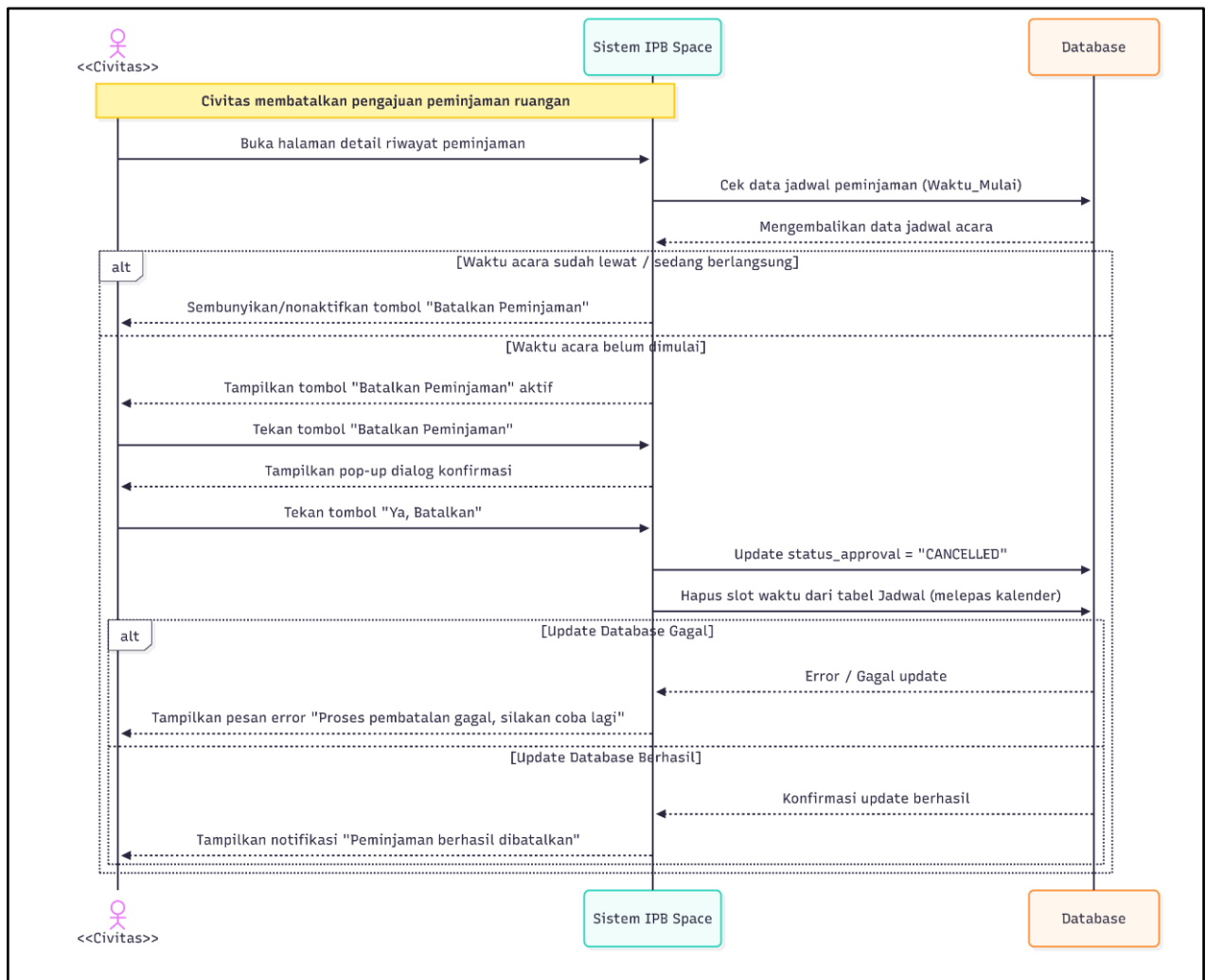
- Melakukan Check-in



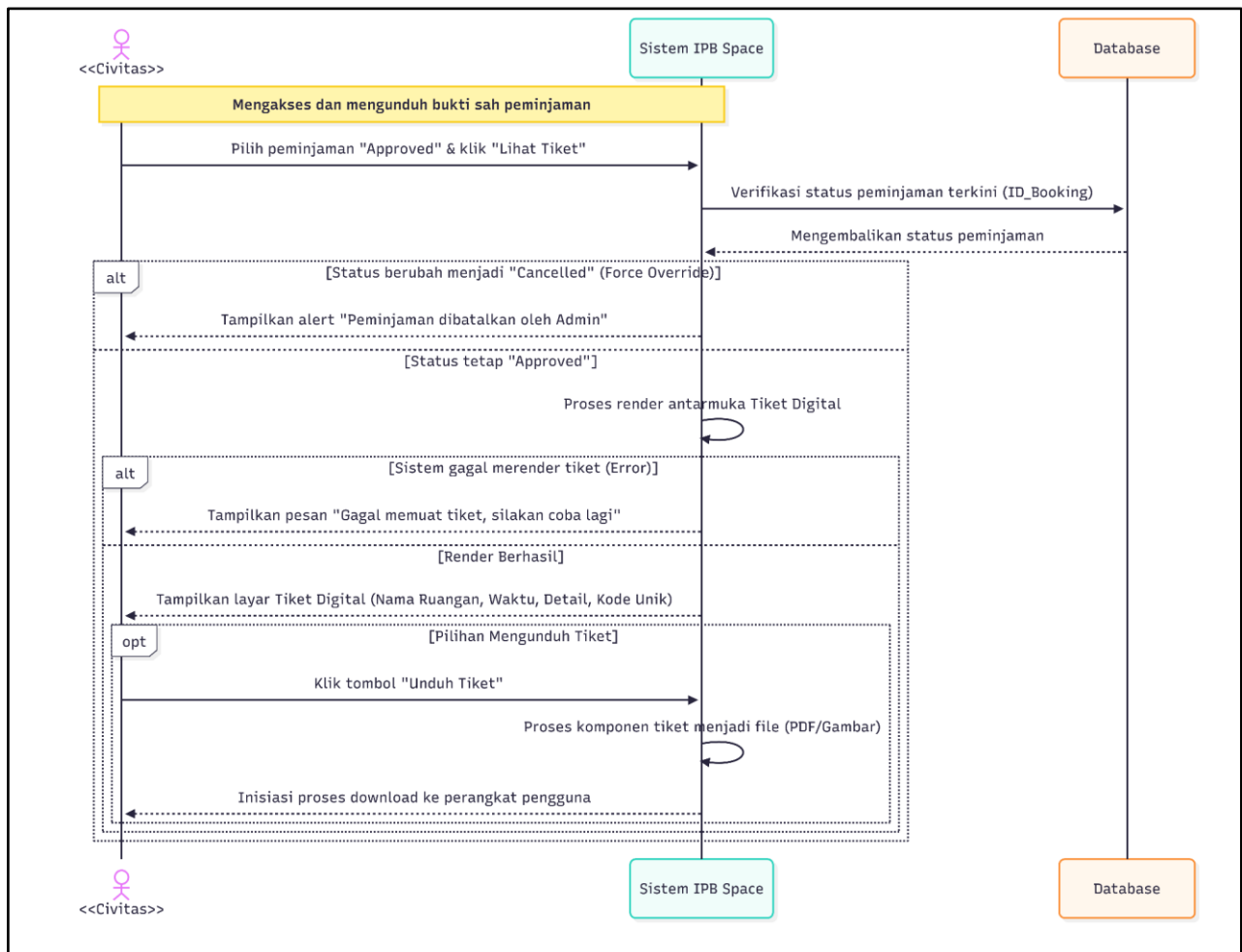
- Memantau Status Peminjaman



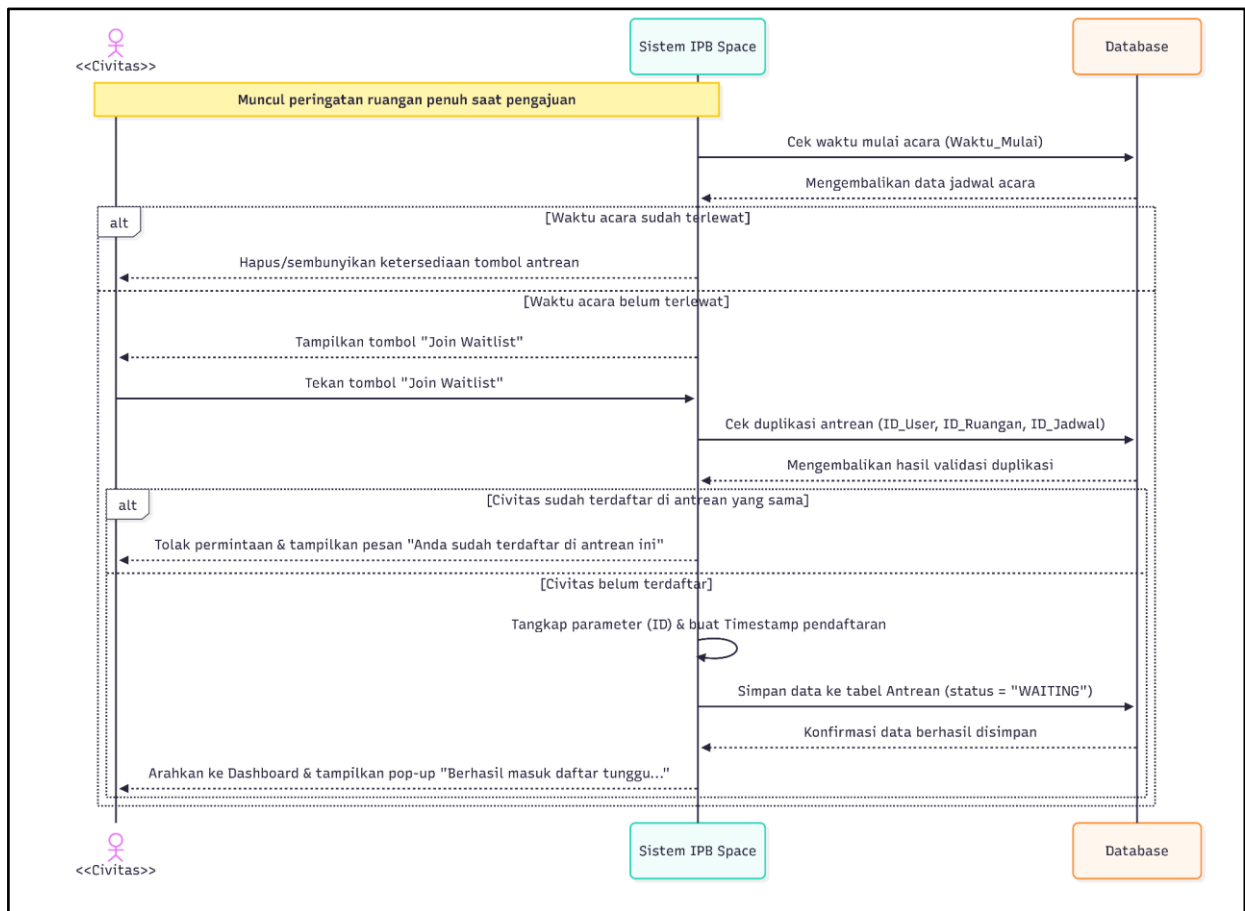
- **Membatalkan Peminjaman**



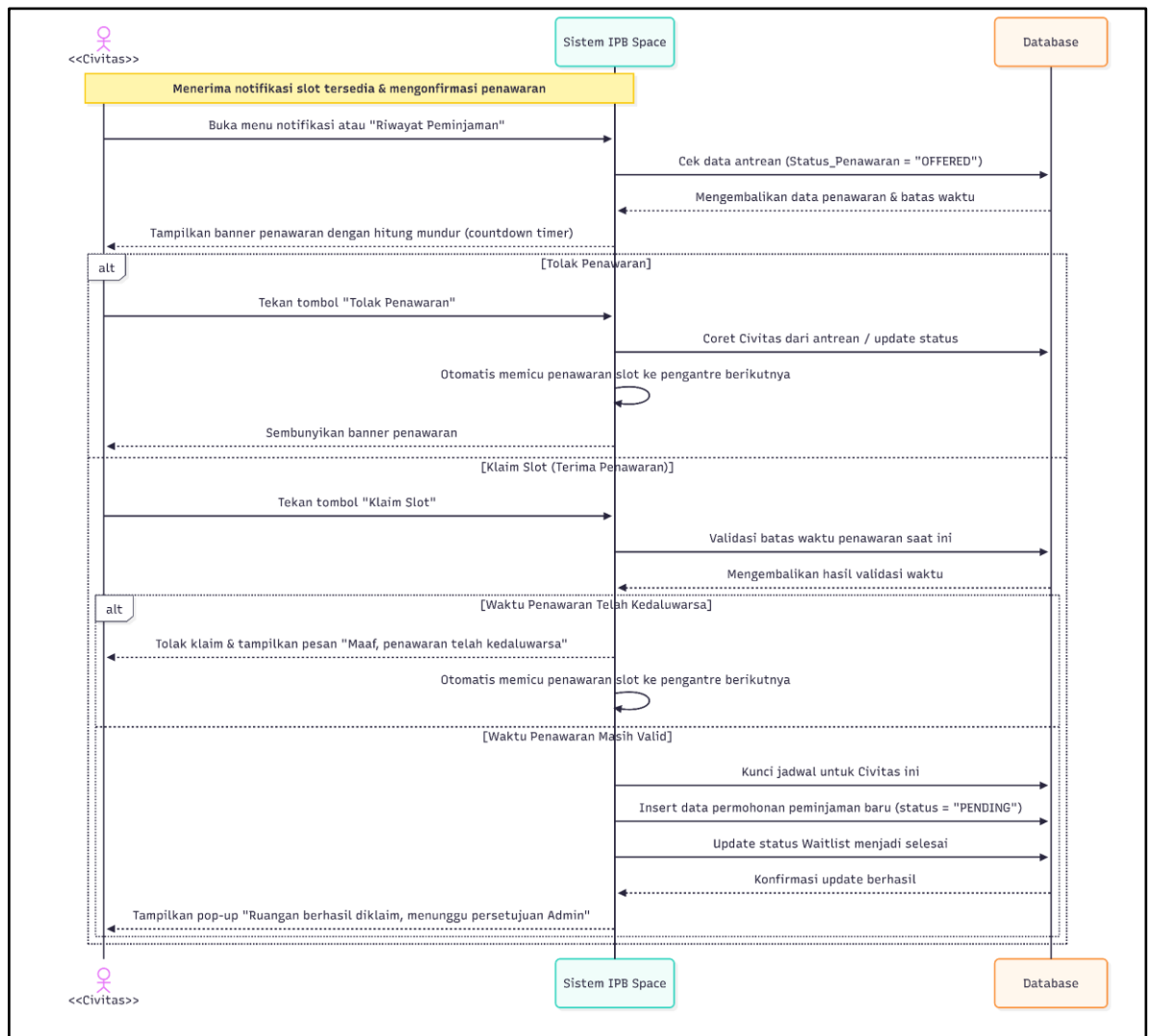
- Mengakses Tiket Digital



- **Masuk ke Daftar Tunggu (Join Waitlist)**



- **Mengonfirmasi Penawaran Antrian**



2. Facility Admin

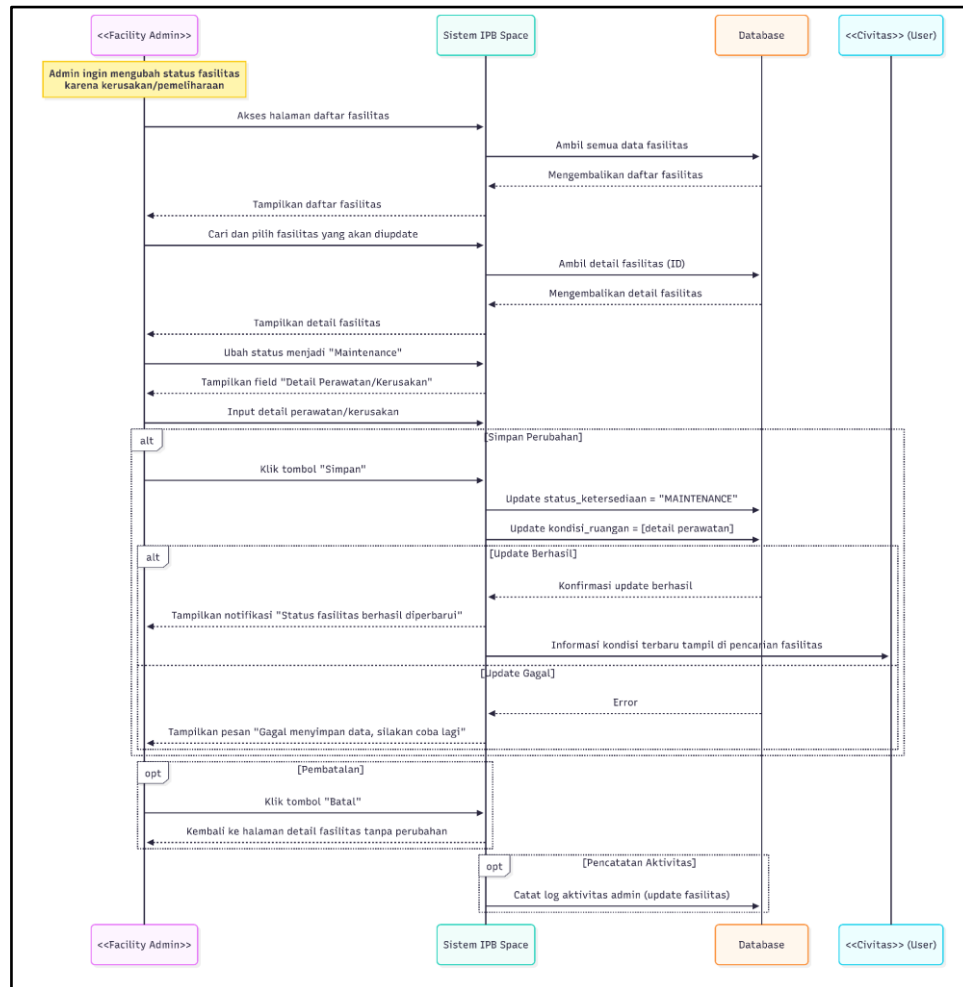
- Memperbarui

Kondisi

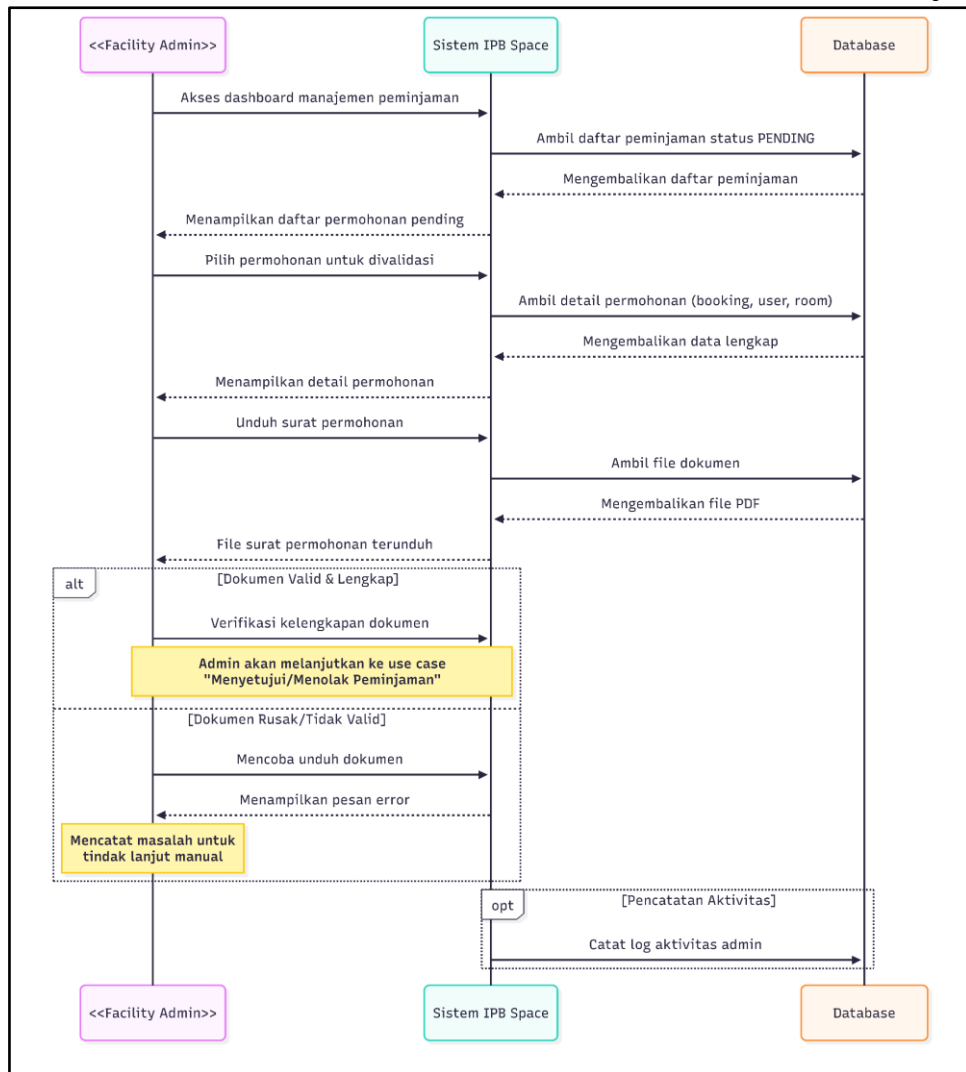
/

Status

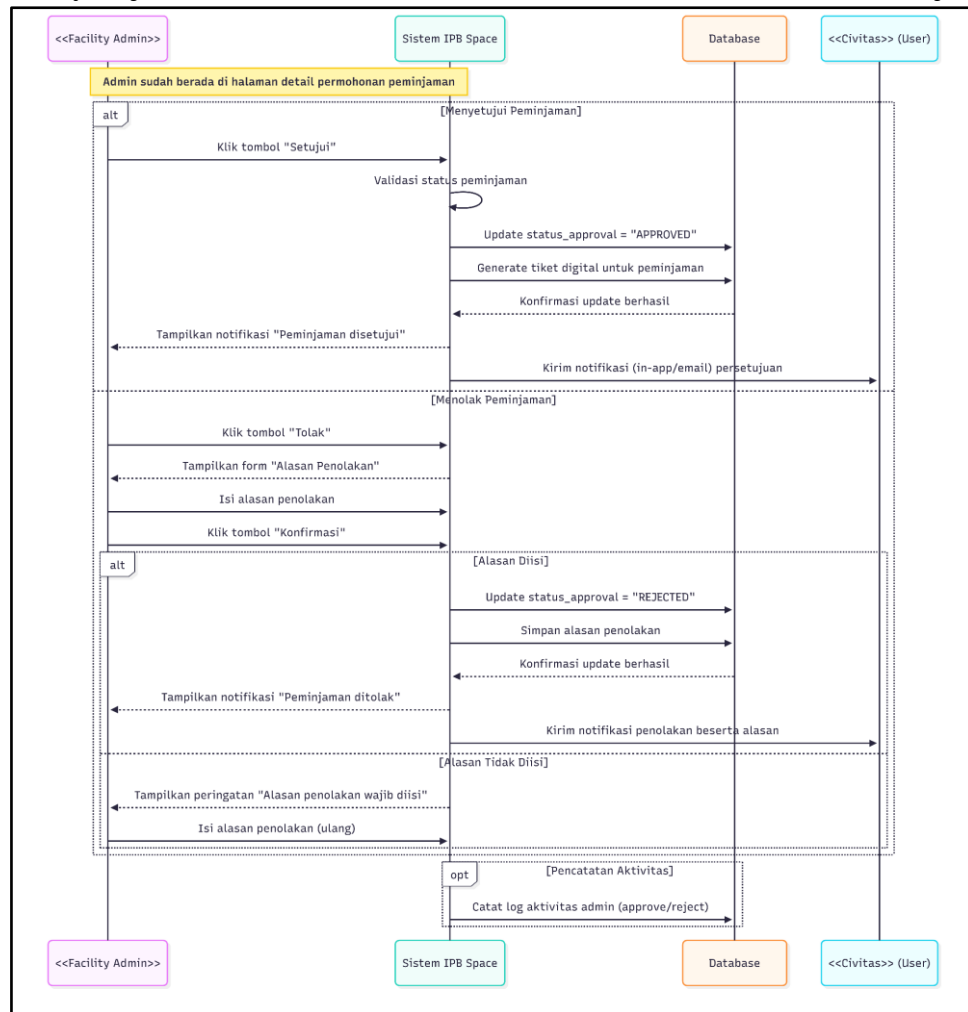
Fasilitas



● Memvalidasi Permohonan Peminjaman

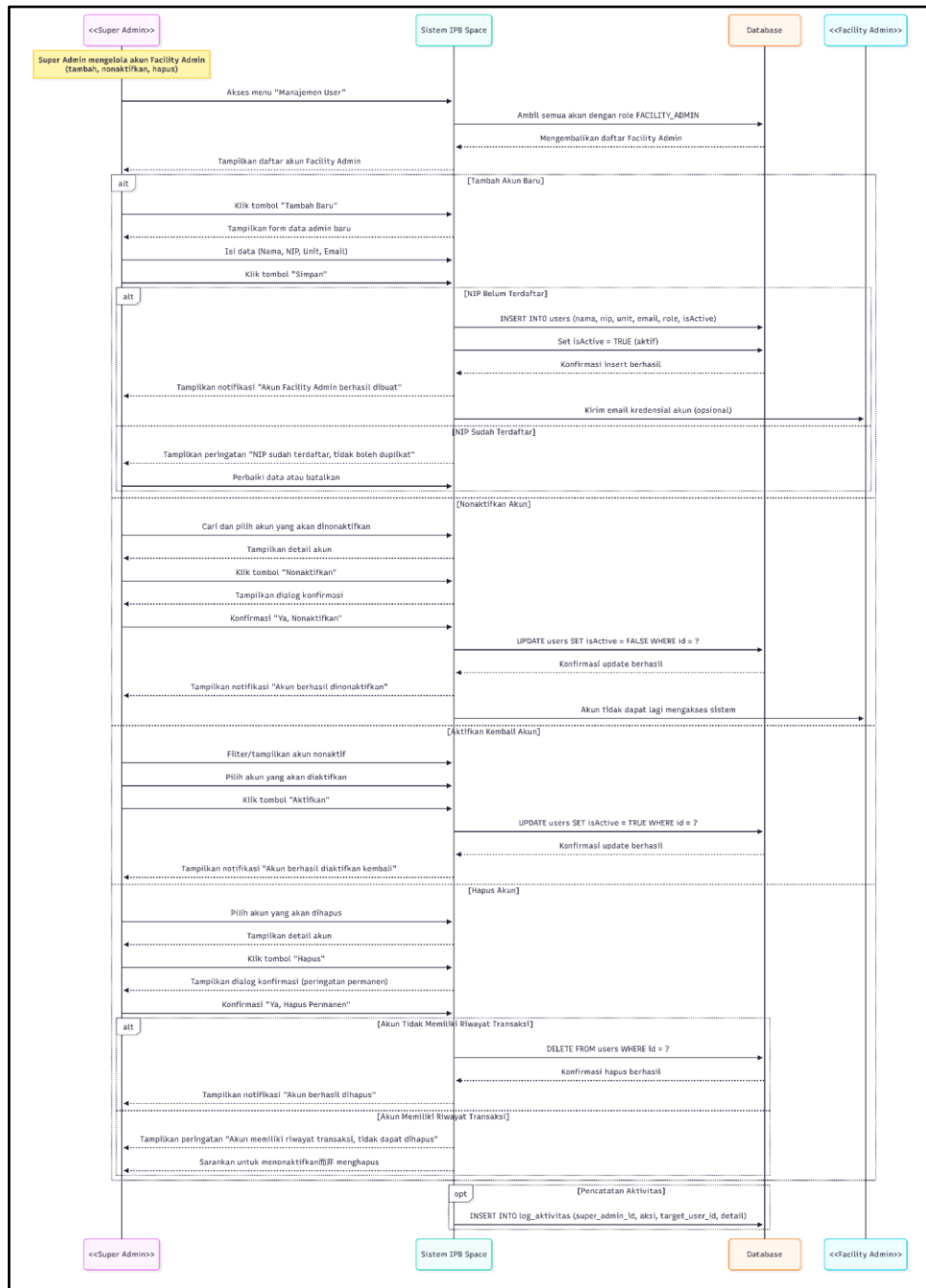


• **Menyetujui / Menolak Peminjaman**

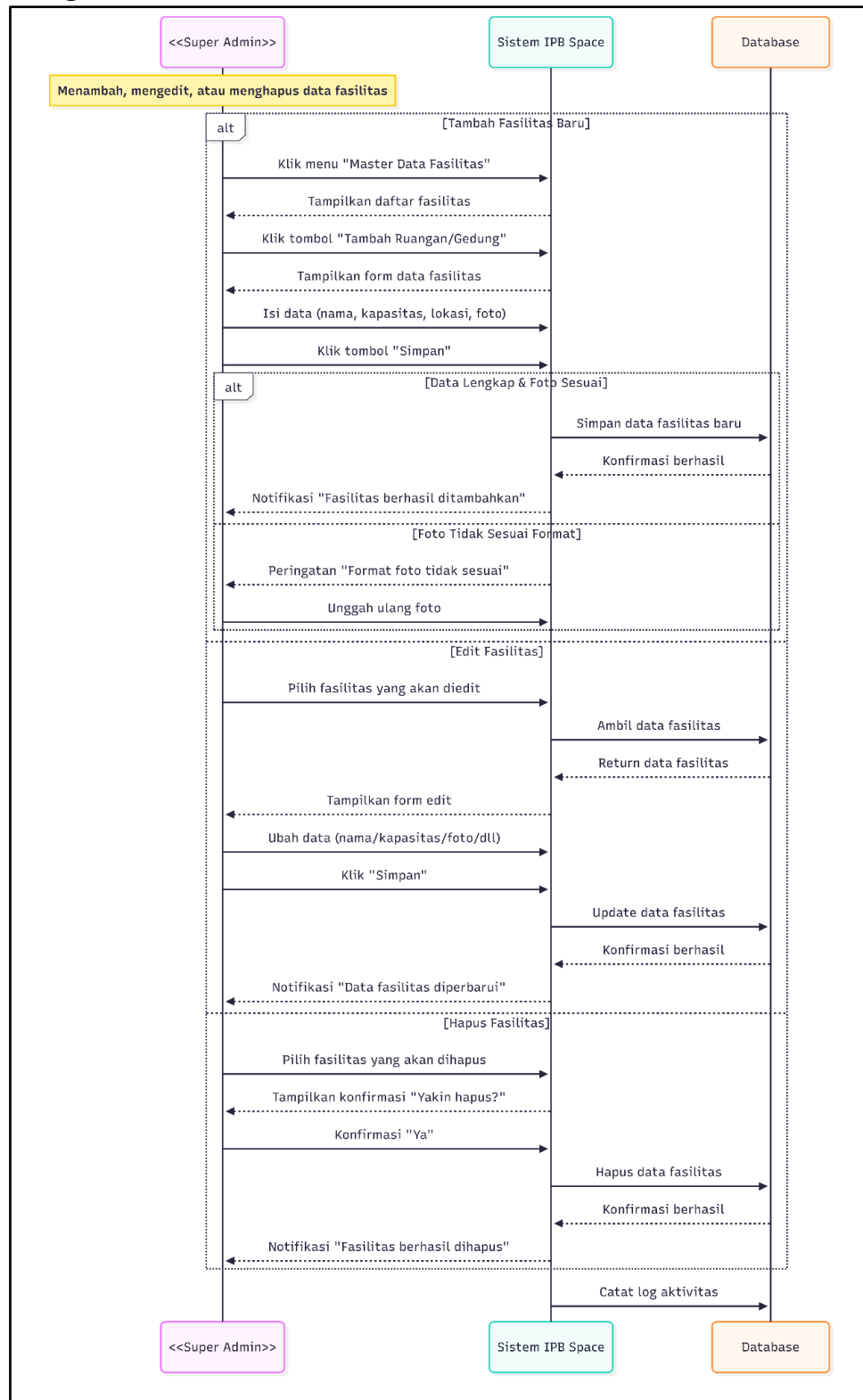


3. Super Admin

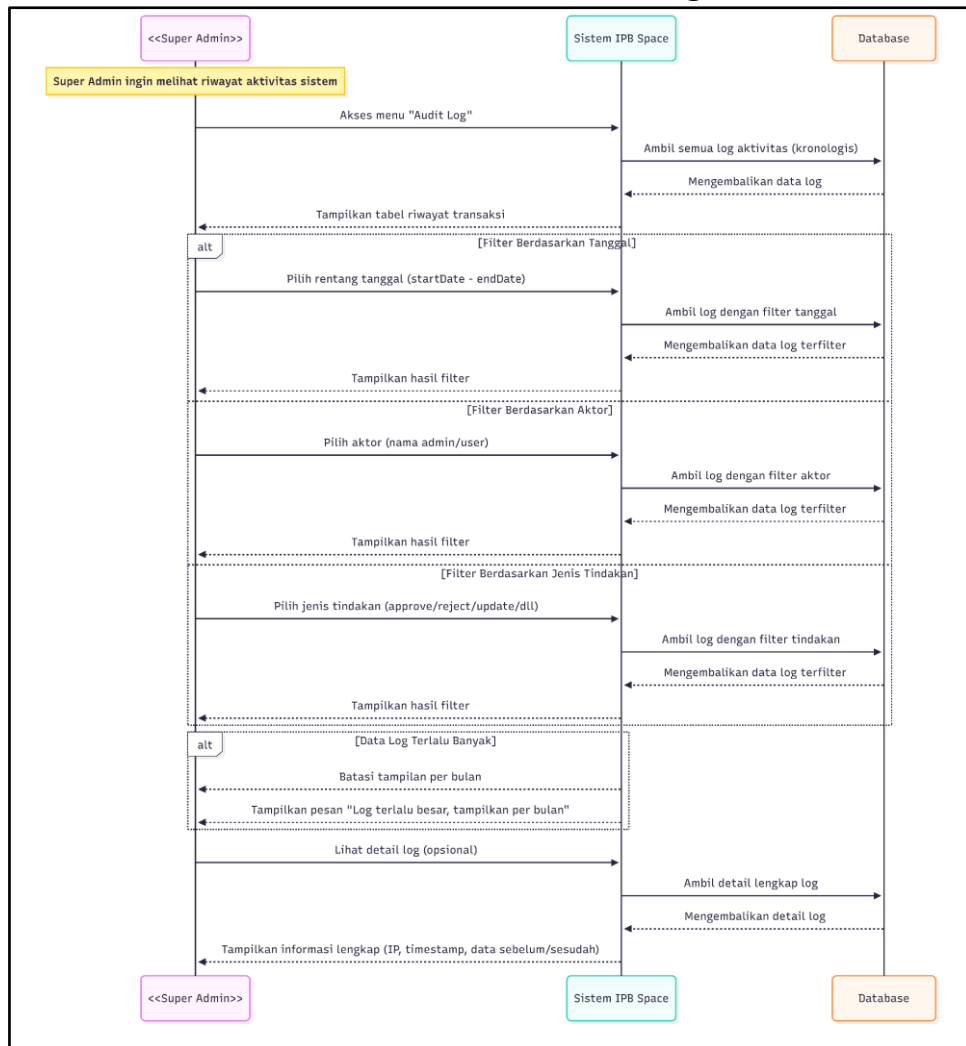
- Mengelola Akun Facility Admin



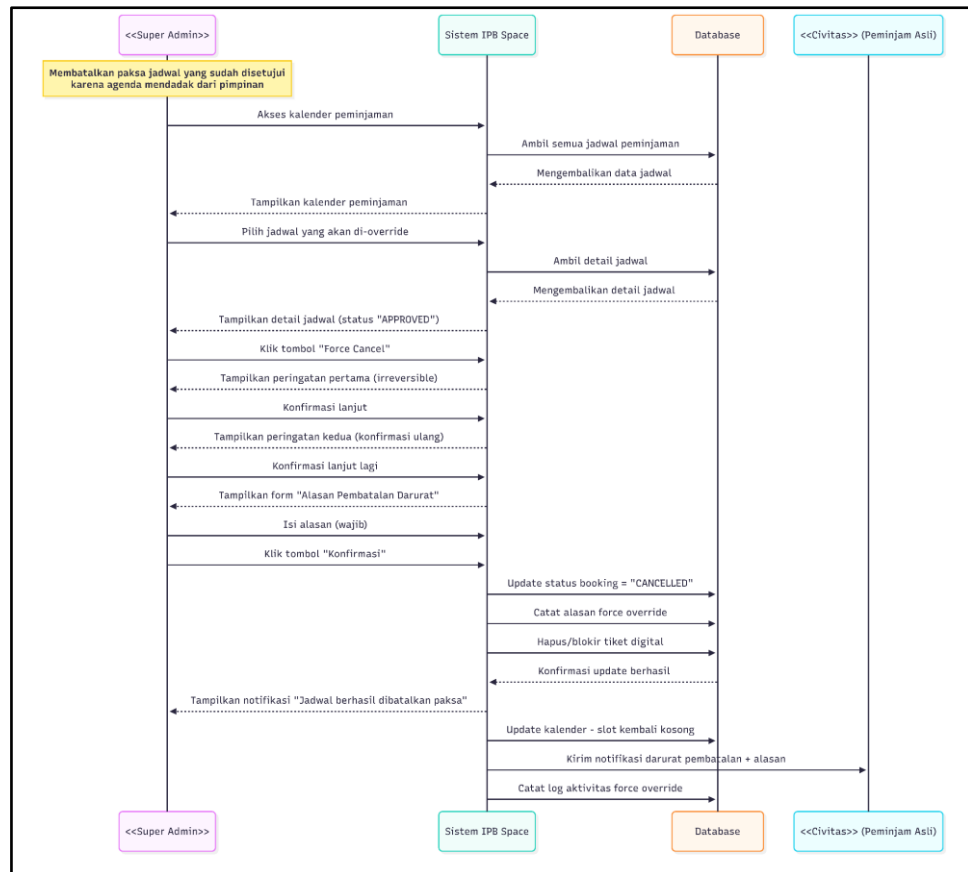
● Mengelola Data Master Fasilitas



● **Melihat** **Audit** **Log** **Sistem**



● Melakukan Force Override Jadwal



4.2 Alur Aliran Otorisasi Peran (*Authorization via RBAC*)

Pembatasan hak akses menu diintegrasikan langsung pada rute API menggunakan injeksi dependensi FastAPI (`backend/app/api/dependencies.py`):

1. Setiap endpoint fungsional diproteksi oleh fungsi dependensi `get_current_user` yang bertugas mendekode dan memverifikasi tanda tangan digital token JWT.
2. Sistem mengimplementasikan utilitas kustom kelas `RoleChecker` yang ditanam pada rute API administratif. Jika klaim token pengguna mengekstrak data role berstatus `Civitas`, sementara endpoint tersebut mendefinisikan batas minimum hak akses peran `Super_Admin` (seperti rute *Force Override*), sistem otomatis memotong alur kerja dan mengembalikan respon kesalahan **403 Forbidden**.

4.3 Alur Aliran Akuntansi Aktivitas (*Accounting & System Audit Log*)

Setiap keputusan operasional kritis yang dieksekusi pada `app/services/booking_service.py` wajib memicu pencatatan akuntansi permanen.

Sistem secara otomatis menangkap data identitas aktor yang lolos dari fungsi otorisasi, menggabungkannya dengan detail nama tabel terdampak, bentuk tindakan administratif (*Approve, Reject, Cancel, Override*), serta stempel waktu (*timestamp*) presisi, untuk kemudian dituliskan secara langsung ke dalam tabel database `Log_Aktivitas`. Karakteristik tabel log ini dikunci agar bersifat *immutable* (tidak dapat dihapus) untuk memitigasi risiko manipulasi jejak digital oleh oknum internal operator.